

Chip in, big win

UVM Sequence 常考题 / 真题汇总

Part 2 / 3 · 题目 9 ~ 16

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 9-16：FIFO 背压统计（题9）、W1C 寄存器验证（题10）、UART 奇偶错误注入（题11）、TLM 多通道路由（题12）、FIFO 溢出/下溢边界（题13）、AXI-S TKEEP（题14）、AXI Outstanding ID 追踪（题15）、寄存器 LOCK 写保护（题16）。

题目 9　FIFO Valid/Ready 背压（Back-pressure）建模与统计

难度：中级
标签：FIFO / backpressure / Valid/Ready / ready_wait / 统计 / 丢包检测

【题目背景】

FIFO 接口普遍采用 Valid/Ready 握手协议： - master 拉高 valid 表示数据就绪； - slave 拉高 ready 表示可以接收； - 只有 valid & ready 同时为 1 时，数据才真正被接收。 当 slave 处理能力不足时会拉低 ready（背压），导致 master 需要等待。 验证目标：统计背压延迟，断言高背压下不发生数据丢失。

【完整要求】

打印 `uvm_warning 并累计 exceed_cnt； 平均等待 = total_wait / num_packets，drop 数，exceed 数；

- 1. 发送 num_packets=30 个数据包，字段随机化；
- 2. 每包发送后通过 get_response 获取 ready_wait 和 dropped；
- 3. 累计 total_wait，若单包 ready_wait > max_wait_thresh（默认 8），
- 4. 若 dropped==1，打印 `uvm_error 并累计 drop_cnt；
- 5. 全部发送完成后打印统计汇报：
- 6. 若 drop_cnt > 0，打印 `uvm_error（背压导致丢包）。

【实现要点】

> - real 类型计算平均值：real'(total_wait) / real'(num_packets)；
> - priority_level 字段为未来扩展（QoS 策略）预留，本题只随机不检查；
> - 本题考查"统计型验证"：不只看单笔对错，更关注宏观指标。

【完整答案代码（含全部类定义）】

```
class fifo_packet_item extends uvm_sequence_item;
    rand bit [31:0] data;
    rand bit [7:0]  tag;
    rand bit [3:0]  priority_level;
         int        ready_wait;   // driver 回填
         bit        dropped;      // driver 回填

    `uvm_object_utils_begin(fifo_packet_item)
        `uvm_field_int(data,           UVM_ALL_ON)
        `uvm_field_int(tag,            UVM_ALL_ON)
        `uvm_field_int(priority_level, UVM_ALL_ON)
        `uvm_field_int(ready_wait,     UVM_ALL_ON)
        `uvm_field_int(dropped,        UVM_ALL_ON)
    `uvm_object_utils_end

    constraint priority_c { priority_level inside {[0:7]}; }

    function new(string name = "fifo_packet_item");
        super.new(name);
    endfunction
endclass

class fifo_backpressure_sequence extends uvm_sequence#(fifo_packet_item);
    `uvm_object_utils(fifo_backpressure_sequence)

    int unsigned num_packets;
    int unsigned max_wait_thresh;

    function new(string name = "fifo_backpressure_sequence");
        super.new(name);
        num_packets     = 30;
        max_wait_thresh = 8;
    endfunction

    task body();
        fifo_packet_item req, rsp;
        int total_wait = 0, drop_cnt = 0, exceed_cnt = 0;

        repeat (num_packets) begin
            req = fifo_packet_item::type_id::create("fifo_pkt");
            start_item(req);
            if (!req.randomize())
                `uvm_fatal("RAND_FAIL","FIFO pkt rand failed")
            finish_item(req);
            get_response(rsp);

            total_wait += rsp.ready_wait;
            if (rsp.dropped) begin
                drop_cnt++;
                `uvm_error("FIFO_BP",
                    $sformatf("Packet DROPPED tag=0x%02h pri=%0d",rsp.tag,rsp.priority_level))
            end
            // 考点: 背压超阈值告警
            if (rsp.ready_wait > max_wait_thresh) begin
                exceed_cnt++;
                `uvm_warning("FIFO_BP",
                    $sformatf("High BP: tag=0x%02h wait=%0d",rsp.tag,rsp.ready_wait))
            end
        end

        `uvm_info("FIFO_BP",
            $sformatf("avg_wait=%.1f drop=%0d exceed=%0d",
                real'(total_wait)/real'(num_packets), drop_cnt, exceed_cnt), UVM_LOW)
        if (drop_cnt > 0)
            `uvm_error("FIFO_BP","Packet loss detected under backpressure!")
    endtask
endclass
```

题目 10　W1C（Write-1-to-Clear）寄存器位语义验证

难度：中级
标签：W1C / write-1-to-clear / STATUS register / side-effect / 寄存器语义

【题目背景】

W1C（Write-1-to-Clear）是硬件状态寄存器的常见访问语义： - 写 1：清除对应位（归零）； - 写 0：无效，不影响对应位； - 读操作：返回当前值，不清除。 典型应用：中断状态寄存器（ISR）。软件读取后写 1 清除已处理的中断， 未处理的中断位保持不变。DUT 实现 W1C 时容易误清除写 0 的位，或 有副作用（同时清除高位），必须精确验证。

【完整要求】

重复 num_rounds=4 轮，每轮： （高 24 位不受影响，无副作用）。

- 1. 写 TRIGGER_ADDR，强制置位 STATUS[3:0]（确保有位需要清除）；
- 2. 读 STATUS_ADDR，记录 status_before；
- 3. 计算 clear_mask = status_before & 32'h0000_00FF（只操作低 8 位）；
- 4. 写 STATUS_ADDR = clear_mask（W1C：写 1 清除已置位的低 8 位）；
- 5. 再次读 STATUS_ADDR，记录 status_after；
- 6. 断言 1：(status_after & clear_mask) == 0（已写 1 的位必须清零）；
- 7. 断言 2：(status_after & ~32'hFF) == (status_before & ~32'hFF)

【实现要点】

> - W1C 的关键语义：写入的是 clear_mask（哪些位为 1 就清哪些），
> - 而不是将寄存器整体写为某个值；
> - 断言 2 检查副作用，是 W1C 测试最容易遗漏的部分。

【完整答案代码（含全部类定义）】

```
class reg_transaction extends uvm_sequence_item;
    typedef enum bit { REG_WRITE, REG_READ } reg_dir_e;

    rand reg_dir_e  dir;
    rand bit [11:0] addr;
    rand bit [31:0] wdata;
    bit      [31:0] rdata;

    `uvm_object_utils_begin(reg_transaction)
        `uvm_field_enum(reg_dir_e, dir, UVM_ALL_ON)
        `uvm_field_int(addr,            UVM_ALL_ON)
        `uvm_field_int(wdata,           UVM_ALL_ON)
        `uvm_field_int(rdata,           UVM_ALL_ON)
    `uvm_object_utils_end

    constraint addr_align_c { addr[1:0] == 2'b00; }

    function new(string name = "reg_transaction");
        super.new(name);
    endfunction
endclass

class reg_w1c_sequence extends uvm_sequence#(reg_transaction);
    `uvm_object_utils(reg_w1c_sequence)

    localparam bit [11:0] STATUS_ADDR  = 12'h010;
    localparam bit [11:0] TRIGGER_ADDR = 12'h014;
    int unsigned num_rounds;

    function new(string name = "reg_w1c_sequence");
        super.new(name);
        num_rounds = 4;
    endfunction

    task do_write(bit [11:0] a, bit [31:0] d);
        reg_transaction req, rsp;
        req = reg_transaction::type_id::create("wr");
        start_item(req);
        req.dir = reg_transaction::REG_WRITE;
        req.addr = a; req.wdata = d;
        finish_item(req); get_response(rsp);
    endtask

    task do_read(bit [11:0] a, output bit [31:0] v);
        reg_transaction req, rsp;
        req = reg_transaction::type_id::create("rd");
        start_item(req);
        req.dir = reg_transaction::REG_READ;
        req.addr = a;
        finish_item(req); get_response(rsp);
        v = rsp.rdata;
    endtask

    task body();
        bit [31:0] status_before, status_after, clear_mask;
        repeat (num_rounds) begin
            do_write(TRIGGER_ADDR, $urandom() | 32'h0000_000F);
            do_read(STATUS_ADDR, status_before);
            `uvm_info("W1C",$sformatf("STATUS before=0x%08h",status_before),UVM_MEDIUM)

            // 考点：只写已置位的低 8 位
            clear_mask = status_before & 32'h0000_00FF;
            do_write(STATUS_ADDR, clear_mask);
            do_read(STATUS_ADDR, status_after);

            if ((status_after & clear_mask) != 0)
                `uvm_error("W1C",
                    $sformatf("W1C failed: bits 0x%08h not cleared",clear_mask))
            // 考点：高位不受影响
            if ((status_after & ~32'h0000_00FF) !== (status_before & ~32'h0000_00FF))
                `uvm_error("W1C","W1C side-effect: unintended bits changed")
            else
```

```
                `uvm_info("W1C","W1C OK",UVM_LOW)
        end
    endtask
endclass
```

题目 11　UART 奇偶校验错误注入与帧错误检测

难度：中级
标签：UART / parity / error injection / frame_err / false alarm / 统计

【题目背景】

UART 帧格式：Start(1) + Data(8) + Parity(1) + Stop(1/2)。 接收方在 Stop 位之前计算奇偶校验，若与帧中 Parity 位不符， 则拉高 parity_err；同时通常伴随 frame_err（帧错误标志）。 验证目标： ① 注入错误时，DUT 必须正确报告 parity_err=1（不漏检）； ② 正常帧时，DUT 不得误报 parity_err=1（无假阳性）。

【完整要求】

若 rsp.frame_err==0 则 `uvm_warning； 验证 rsp.rx_data == req.payload，否则 `uvm_error；

- 1. 所有帧均使能奇偶校验（parity_en==1 约束）；
- 2. inject_err 的分布约束：dist { 0:=80, 1:=20 }（约 20% 错误率）；
- 3. 只有 parity_en==1 时才允许 inject_err==1；
- 4. 发送 num_frames=30 帧，对每帧的响应：
- inject_err==1：断言 rsp.parity_err==1，否则 `uvm_error；
- inject_err==0：断言 rsp.parity_err==0（无假阳性），否则 `uvm_error；
- 5. 最终打印统计：good_cnt / err_cnt / false_alarm。

【实现要点】

> - driver 负责按 inject_err 决定是否翻转 parity bit，序列层只产生意图；
> - false_alarm 计数是关键指标，确认 DUT 不会在正常帧上误报。

【完整答案代码（含全部类定义）】

```
class uart_transaction extends uvm_sequence_item;
    rand bit [7:0] payload;
    rand bit       parity_en;
    rand bit       parity_type;
    rand bit       inject_err;   // 考点: 错误注入控制
    rand bit [1:0] stop_bits;
         bit [7:0] rx_data;
         bit       parity_err;
         bit       frame_err;

    `uvm_object_utils_begin(uart_transaction)
        `uvm_field_int(payload,     UVM_ALL_ON)
        `uvm_field_int(parity_en,   UVM_ALL_ON)
        `uvm_field_int(parity_type, UVM_ALL_ON)
        `uvm_field_int(inject_err,  UVM_ALL_ON)
        `uvm_field_int(stop_bits,   UVM_ALL_ON)
        `uvm_field_int(rx_data,     UVM_ALL_ON)
        `uvm_field_int(parity_err,  UVM_ALL_ON)
        `uvm_field_int(frame_err,   UVM_ALL_ON)
    `uvm_object_utils_end

    constraint stop_bits_c  { stop_bits inside {2'b01, 2'b10}; }
    constraint err_inject_c { !parity_en -> inject_err == 0; }
    constraint err_rate_c   { inject_err dist { 0:=80, 1:=20 }; }

    function new(string name = "uart_transaction");
        super.new(name);
    endfunction
endclass

class uart_parity_error_sequence extends uvm_sequence#(uart_transaction);
    `uvm_object_utils(uart_parity_error_sequence)

    int unsigned num_frames;
    int          good_cnt, err_cnt, false_alarm;

    function new(string name = "uart_parity_error_sequence");
        super.new(name);
        num_frames = 30;
    endfunction

    task body();
        uart_transaction req, rsp;
        good_cnt = 0; err_cnt = 0; false_alarm = 0;

        repeat (num_frames) begin
            req = uart_transaction::type_id::create("uart_frm");
            start_item(req);
            if (!req.randomize() with { parity_en == 1; })
                `uvm_fatal("RAND_FAIL","UART rand failed")
            finish_item(req);
            get_response(rsp);

            if (req.inject_err) begin
                err_cnt++;
                if (!rsp.parity_err)
                    `uvm_error("UART_SEQ",
                        $sformatf("Parity error not detected! data=0x%02h",req.payload))
                if (!rsp.frame_err)
                    `uvm_warning("UART_SEQ","frame_err not set on corrupted frame")
            end else begin
                good_cnt++;
                if (rsp.parity_err) begin
                    false_alarm++;
                    `uvm_error("UART_SEQ",
                        $sformatf("False parity error: data=0x%02h",req.payload))
                end
                if (!rsp.parity_err && rsp.rx_data !== req.payload)
                    `uvm_error("UART_SEQ",
                        $sformatf("Data error: sent=0x%02h got=0x%02h",
                            req.payload,rsp.rx_data))
            end
```

```
        end
        `uvm_info("UART_SEQ",
            $sformatf("Done: good=%0d err=%0d false_alarm=%0d",
                good_cnt,err_cnt,false_alarm), UVM_LOW)
    endtask
endclass
```

题目 12　TLM 多通道路由无串扰验证

难度：中级
标签：TLM / channel routing / crosstalk / echo / 多通道

【题目背景】

DUT 内部有一个 4 路 TLM（Transaction Level Modeling）通道路由器， 将输入数据包按 channel 字段路由到对应的输出端口，再由 driver 通过 TLM FIFO 回传给 sequence。 验证目标：确保通道路由正确，不同通道之间数据不互相干扰（无串扰）。

【完整要求】

- 1. 外层 for(ch=0..3) 循环，每个通道发送 num_per_channel=8 笔数据；
- 2. inline constraint 强制 channel == ch，确保该轮只发指定通道；
- 3. 通过 get_response(rsp) 获取 echoed 和 echo_ch；
- 4. 断言 A：rsp.echoed == req.payload（数据无损）；
- 5. 断言 B：rsp.echo_ch == req.channel（通道编号正确，无串扰）；
- 6. 任一断言失败则 `uvm_error 打印通道号和期望/实际值。

【实现要点】

> - 本题考查真实的 TLM 路由语义，而非简单的 put/get 调用；
> - echo_ch 验证是串扰检测的核心：即使数据值恰好一致，
> - 但通道号错误同样是严重 bug；
> - 按通道分批（而非混合随机）发送，便于隔离问题。

【完整答案代码（含全部类定义）】

```
class tlm_data_item extends uvm_sequence_item;
    rand bit [63:0] payload;
    rand bit [3:0]  channel;
         bit [63:0] echoed;
         bit [3:0]  echo_ch;   // driver 回填

    `uvm_object_utils_begin(tlm_data_item)
        `uvm_field_int(payload,  UVM_ALL_ON)
        `uvm_field_int(channel,  UVM_ALL_ON)
        `uvm_field_int(echoed,   UVM_ALL_ON)
        `uvm_field_int(echo_ch,  UVM_ALL_ON)
    `uvm_object_utils_end

    constraint channel_c { channel inside {[0:3]}; }

    function new(string name = "tlm_data_item");
        super.new(name);
    endfunction
endclass

class tlm_channel_routing_sequence extends uvm_sequence#(tlm_data_item);
    `uvm_object_utils(tlm_channel_routing_sequence)

    int unsigned num_per_channel;

    function new(string name = "tlm_channel_routing_sequence");
        super.new(name);
        num_per_channel = 8;
    endfunction

    task body();
        tlm_data_item req, rsp;
        // 考点: 按 channel 分批, 验证路由无串扰
        for (int ch = 0; ch < 4; ch++) begin
            `uvm_info("TLM_SEQ",$sformatf("=== Channel %0d ===",ch),UVM_LOW)
            repeat (num_per_channel) begin
                req = tlm_data_item::type_id::create("tlm_item");
                start_item(req);
                if (!req.randomize() with { channel == ch; })
                    `uvm_fatal("RAND_FAIL","TLM rand failed")
                finish_item(req);
                get_response(rsp);

                if (rsp.echoed !== req.payload)
                    `uvm_error("TLM_SEQ",
                        $sformatf("ch%0d data mismatch: sent=0x%016h got=0x%016h",
                            ch,req.payload,rsp.echoed))
                if (rsp.echo_ch !== req.channel)
                    `uvm_error("TLM_SEQ",
                        $sformatf("Channel crosstalk! sent=%0d got=%0d",
                            req.channel,rsp.echo_ch))
            end
        end
    endtask
endclass
```

题目 13　FIFO 满/空边界条件 + 溢出/下溢保护验证

难度：中级
标签：FIFO / overflow / underflow / boundary / full / empty / 数据完整性

【题目背景】

FIFO 控制器的三个关键边界条件是最常见的硬件 bug 来源： ① 溢出（overflow）：写满后继续写，DUT 应拒绝并置 overflow 标志； ② 下溢（underflow）：读空后继续读，DUT 应拒绝并置 underflow 标志； ③ 数据完整性：写入的数据经过 FIFO 后读出，顺序和内容不变。

【完整要求】

阶段 1（写满）：循环 FIFO_DEPTH 次写入，记录写入数据到 wr_log[$]； 在第 FIFO_DEPTH 次写入后检查 rsp.full，若为 0 则 `uvm_error； 阶段 2（测试溢出）：写满后再发一次 FIFO_WRITE； 断言 rsp.overflow == 1，失败则 `uvm_error； 成功则 `uvm_info "Overflow correctly detected"； 阶段 3（读空并校验顺序）：循环 FIFO_DEPTH 次读取； 每次读取比对 rsp.data 与 wr_log[i]，不一致则 `uvm_error； 阶段 4（测试下溢）：读空后再发一次 FIFO_READ； 断言 rsp.underflow == 1，失败则 `uvm_error。

【实现要点】

- wr_log 是动态数组（bit[31:0] wr_log[$]），用 push_back 记录写入序列；
- 顺序校验是 FIFO 最重要的语义：先进先出，读出顺序必须与写入顺序一致；
- overflow/underflow 是保护机制，验证 DUT 不会悄无声息地丢数据或给出垃圾数据。

【完整答案代码（含全部类定义）】

```
class fifo_rw_item extends uvm_sequence_item;
    typedef enum bit { FIFO_WRITE, FIFO_READ } fifo_dir_e;

    rand fifo_dir_e dir;
    rand bit [31:0] data;
    bit             full, empty, overflow, underflow;

    `uvm_object_utils_begin(fifo_rw_item)
        `uvm_field_enum(fifo_dir_e, dir, UVM_ALL_ON)
        `uvm_field_int(data,             UVM_ALL_ON)
        `uvm_field_int(full,             UVM_ALL_ON)
        `uvm_field_int(empty,            UVM_ALL_ON)
        `uvm_field_int(overflow,         UVM_ALL_ON)
        `uvm_field_int(underflow,        UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "fifo_rw_item");
        super.new(name);
    endfunction
endclass

class fifo_boundary_sequence extends uvm_sequence#(fifo_rw_item);
    `uvm_object_utils(fifo_boundary_sequence)

    localparam int FIFO_DEPTH = 16;

    function new(string name = "fifo_boundary_sequence");
        super.new(name);
    endfunction

    task body();
        fifo_rw_item req, rsp;
        bit [31:0] wr_log[$];

        // 写满
        for (int i = 0; i < FIFO_DEPTH; i++) begin
            req = fifo_rw_item::type_id::create("wr");
            start_item(req);
            if (!req.randomize() with { dir == fifo_rw_item::FIFO_WRITE; })
                `uvm_fatal("RAND_FAIL","FIFO wr rand failed")
            finish_item(req); get_response(rsp);
            wr_log.push_back(req.data);
            if (i == FIFO_DEPTH-1 && !rsp.full)
                `uvm_error("FIFO_BND","full flag not set after DEPTH writes")
        end

        // 考点①: 满时再写 → overflow=1
        req = fifo_rw_item::type_id::create("overflow");
        start_item(req);
        if (!req.randomize() with { dir == fifo_rw_item::FIFO_WRITE; })
            `uvm_fatal("RAND_FAIL","overflow rand failed")
        finish_item(req); get_response(rsp);
        if (!rsp.overflow)
            `uvm_error("FIFO_BND","Overflow not flagged on write-when-full")
        else
            `uvm_info("FIFO_BND","Overflow correctly detected",UVM_MEDIUM)

        // 读空，校验数据顺序
        for (int i = 0; i < FIFO_DEPTH; i++) begin
            req = fifo_rw_item::type_id::create("rd");
            start_item(req);
            req.dir = fifo_rw_item::FIFO_READ;
            finish_item(req); get_response(rsp);
            if (rsp.data !== wr_log[i])
                `uvm_error("FIFO_BND",
                    $sformatf("Order mismatch idx=%0d exp=0x%08h got=0x%08h",
                        i,wr_log[i],rsp.data))
        end

        // 考点②: 空时再读 → underflow=1
        req = fifo_rw_item::type_id::create("underflow");
        start_item(req);
```

```
        req.dir = fifo_rw_item::FIFO_READ;
        finish_item(req); get_response(rsp);
        if (!rsp.underflow)
            `uvm_error("FIFO_BND","Underflow not flagged on read-when-empty")
        else
            `uvm_info("FIFO_BND","Underflow correctly detected",UVM_MEDIUM)
    endtask
endclass
```

题目 14　AXI-Stream TKEEP 末尾字节使能 + EOP 检测

难度：中级
标签：AXI-Stream / TKEEP / TSTRB / TLAST / EOP / 字节使能

【题目背景】

AXI-Stream 协议使用 4 字节宽数据总线（32-bit），每拍由 TKEEP 标记 哪些字节携带有效数据。对于非末尾拍，所有字节均有效（TKEEP=4'b1111）； 对于末尾拍（TLAST=1），包的长度可能不是 4 的整数倍，此时 TKEEP 的低若干位为 1（有效），高位为 0（填充）。 TSTRB 是 TKEEP 的子集，标记"位置字节"（position byte）。

【完整要求】

- 1. tkeep 约束：
- 非末尾拍（tlast==0）： tkeep 必须为 4'b1111；
- 末尾拍（tlast==1）： tkeep 只允许 0001/0011/0111/1111（连续低位有效）；
- 2. tstrb 约束：(tstrb & ~tkeep) == 0（tstrb 不能超出 tkeep 范围）；
- 3. 随机化包长 pkt_len 在 [min_len, max_len] 范围内；
- 4. 每包的最后一拍强制 tlast==1，其余拍 tlast==0（inline constraint）；
- 5. 每拍收到响应后检查：
- 非末尾拍 tkeep!=4'b1111 → `uvm_error；
- 末尾拍 tkeep==4'b0000 → `uvm_error；
- 6. 统计并打印每包的拍数和有效字节总数（valid_bytes 函数）。

【实现要点】

> - TKEEP 连续有效位约束（低位到高位）是 AXI-Stream 规范的明确要求；
> - valid_bytes() 函数：遍历 tkeep 每位，计算有效字节数；
> - 末尾拍用 inline constraint: tlast == (beat == pkt_len - 1)。

【完整答案代码（含全部类定义）】

```
class axis_transaction extends uvm_sequence_item;
    rand bit [31:0] tdata;
    rand bit [3:0]  tkeep;
    rand bit [3:0]  tstrb;
    rand bit        tlast;
    rand bit [7:0]  tuser;

    `uvm_object_utils_begin(axis_transaction)
        `uvm_field_int(tdata, UVM_ALL_ON)
        `uvm_field_int(tkeep, UVM_ALL_ON)
        `uvm_field_int(tstrb, UVM_ALL_ON)
        `uvm_field_int(tlast, UVM_ALL_ON)
        `uvm_field_int(tuser, UVM_ALL_ON)
    `uvm_object_utils_end

    // 考点: TKEEP 约束
    constraint tkeep_mid_c  { !tlast -> tkeep == 4'b1111; }
    constraint tkeep_last_c { tlast  -> tkeep inside {4'b0001,4'b0011,4'b0111,4'b1111}; }
    constraint tstrb_c      { (tstrb & ~tkeep) == 0; }

    function new(string name = "axis_transaction");
        super.new(name);
    endfunction

    function int valid_bytes();
        int cnt = 0;
        for (int i = 0; i < 4; i++) if (tkeep[i]) cnt++;
        return cnt;
    endfunction
endclass

class axis_tkeep_sequence extends uvm_sequence#(axis_transaction);
    `uvm_object_utils(axis_tkeep_sequence)

    int unsigned num_packets, min_len, max_len;

    function new(string name = "axis_tkeep_sequence");
        super.new(name);
        num_packets=8; min_len=3; max_len=10;
    endfunction

    task body();
        axis_transaction req, rsp;
        int pkt_len, total_bytes;

        repeat (num_packets) begin
            void'(std::randomize(pkt_len) with { pkt_len>=min_len; pkt_len<=max_len; });
            total_bytes = 0;
            for (int beat = 0; beat < pkt_len; beat++) begin
                req = axis_transaction::type_id::create("axis_beat");
                start_item(req);
                if (!req.randomize() with { tlast == (beat == pkt_len-1); })
                    `uvm_fatal("RAND_FAIL","AXIS rand failed")
                finish_item(req); get_response(rsp);
                total_bytes += req.valid_bytes();

                if (!req.tlast && req.tkeep != 4'b1111)
                    `uvm_error("AXIS_SEQ",
                        $sformatf("Mid-packet tkeep!=0xF at beat%0d",beat))
                if (req.tlast && req.tkeep == 4'b0000)
                    `uvm_error("AXIS_SEQ","TLAST beat has tkeep=0!")
            end
            `uvm_info("AXIS_SEQ",
                $sformatf("Pkt done: beats=%0d bytes=%0d",pkt_len,total_bytes),UVM_MEDIUM)
        end
    endtask
endclass
```

题目 15　AXI4 Outstanding 事务管理（AWID/BID ID 追踪）

难度：高级
标签：AXI4 / outstanding / AWID / BID / out-of-order / 关联数组

【题目背景】

AXI4 协议支持 outstanding（飞行中）事务：master 可以在收到前一笔 写响应之前发出下一笔写请求（流水线化），从而提高总线利用率。 此外，AXI4 允许 out-of-order 响应：B 通道的响应可以以不同于请求 的顺序返回（BID 可能与 AWID 发出顺序不同）。 验证难点：如何正确匹配乱序返回的 BID 与对应的原始请求数据?

【完整要求】

若存在则 `uvm_error（未归还前不得重用）； 否则报告未收到响应的事务数。

- 1. 定义关联数组 pending_data[bit[3:0]]，键为 AWID，值为对应 wdata；
- 2. 发出请求前：检查 pending_data 中是否已存在相同 AWID，
- 3. 将 req.awid 作为键，req.wdata 为值存入 pending_data；
- 4. 发送事务并 get_response；
- 5. 用 rsp.bid 在 pending_data 中查表，找到对应 wdata：
- 找到则打印 `uvm_info，并从表中删除该条目；
- 未找到则 `uvm_error "Unexpected BID"；
- 6. 检查 bresp==2'b00（SC），否则 `uvm_error；
- 7. num_trans 笔全部完成后，检查 pending_data.size()==0，

【实现要点】

> - 关联数组 pending_data[bit[3:0]] 是 SystemVerilog 特有语法；
> - delete() 方法从关联数组中移除条目；exists() 方法检查键是否存在；
> - 本题模拟的是半 outstanding（每发一笔等一笔响应），
> - 完整 outstanding 需要双线程（发送线程 + 接收线程 fork）。

【完整答案代码（含全部类定义）】

```
class axi_wr_transaction extends uvm_sequence_item;
    rand bit [3:0]  awid;
    rand bit [31:0] awaddr;
    rand bit [7:0]  awlen;
    rand bit [31:0] wdata;
    rand bit [3:0]  wstrb;
    bit      [3:0]  bid;     // driver 回填
    bit      [1:0]  bresp;

    `uvm_object_utils_begin(axi_wr_transaction)
        `uvm_field_int(awid,   UVM_ALL_ON)
        `uvm_field_int(awaddr, UVM_ALL_ON)
        `uvm_field_int(awlen,  UVM_ALL_ON)
        `uvm_field_int(wdata,  UVM_ALL_ON)
        `uvm_field_int(wstrb,  UVM_ALL_ON)
        `uvm_field_int(bid,    UVM_ALL_ON)
        `uvm_field_int(bresp,  UVM_ALL_ON)
    `uvm_object_utils_end

    constraint addr_align_c { awaddr[1:0] == 2'b00; }
    constraint len_c        { awlen inside {8'd0, 8'd3, 8'd7}; }
    constraint wstrb_c      { wstrb != 4'b0000; }

    function new(string name = "axi_wr_transaction");
        super.new(name);
    endfunction
endclass

class axi_outstanding_sequence extends uvm_sequence#(axi_wr_transaction);
    `uvm_object_utils(axi_outstanding_sequence)

    int unsigned num_trans;

    function new(string name = "axi_outstanding_sequence");
        super.new(name);
        num_trans = 16;
    endfunction

    task body();
        axi_wr_transaction req, rsp;
        // 考点: 关联数组追踪 ID→wdata
        bit [31:0] pending_data[bit[3:0]];

        for (int i = 0; i < num_trans; i++) begin
            req = axi_wr_transaction::type_id::create("axi_wr");
            start_item(req);
            if (!req.randomize())
                `uvm_fatal("RAND_FAIL","AXI outstanding rand failed")
            // 考点: 发出前确保 ID 不重复
            if (pending_data.exists(req.awid))
                `uvm_error("OUTSTANDING",
                    $sformatf("Duplicate AWID=0x%0h issued before response",req.awid))
            pending_data[req.awid] = req.wdata;
            finish_item(req);
            get_response(rsp);

            // 考点: 用 BID 查表验证
            if (rsp.bresp != 2'b00)
                `uvm_error("OUTSTANDING",
                    $sformatf("BRESP error: BID=%0h bresp=%02b",rsp.bid,rsp.bresp))
            if (pending_data.exists(rsp.bid)) begin
                `uvm_info("OUTSTANDING",
                    $sformatf("BID=0x%0h OK wdata=0x%08h",rsp.bid,pending_data[rsp.bid]),
                    UVM_HIGH)
                pending_data.delete(rsp.bid);
            end else
                `uvm_error("OUTSTANDING",
                    $sformatf("Unexpected BID=0x%0h",rsp.bid))
        end

        if (pending_data.size() != 0)
            `uvm_error("OUTSTANDING",
```

```
                $sformatf("%0d transactions never got response!",pending_data.size()))
    endtask
endclass
```

题目 16　寄存器 LOCK 写保护位验证

难度：中级
标签：register / LOCK bit / write protect / 软件参考模型 / 三步校验

【题目背景】

安全敏感的寄存器（如时钟配置、安全基地址）通常配套一个 LOCK 控制寄存器。 一旦 LOCK 寄存器被写入 1，受保护的数据寄存器变为只读，直到系统复位。　这防止了运行时对关键配置的意外修改。　验证目标：锁定后写入被忽略，解锁后写入恢复正常。

【完整要求】

Step 1（锁定前写入）： Step 2（锁定后写入，期望被忽略）： Step 3（解锁后写入）：

- 写 LOCK_ADDR=0（确保未锁定）；
- 写 DATA_ADDR=0xDEAD_BEEF；
- 读 DATA_ADDR，断言 rdata==0xDEAD_BEEF；
- 写 LOCK_ADDR=1（锁定）；
- 写 DATA_ADDR=0xCAFE_BABE（应被 DUT 拒绝）；
- 读 DATA_ADDR，断言 rdata==0xDEAD_BEEF（值未变）；
- 写 LOCK_ADDR=0（解锁）；
- 写 DATA_ADDR=0x1234_5678；
- 读 DATA_ADDR，断言 rdata==0x1234_5678。

【实现要点】

> - do_write / do_read 辅助 task 封装重复的 start_item/finish_item 逻辑；
> - Step 2 是核心验证点：LOCK=1 时 DUT 不应修改 DATA 寄存器；
> - 三步验证覆盖了"写前"、"锁定期间"、"解锁后"三种状态。

【完整答案代码（含全部类定义）】

```
class reg_lock_sequence extends uvm_sequence#(reg_transaction);
    `uvm_object_utils(reg_lock_sequence)

    localparam bit [11:0] LOCK_ADDR = 12'h000;
    localparam bit [11:0] DATA_ADDR = 12'h004;

    function new(string name = "reg_lock_sequence");
        super.new(name);
    endfunction

    task do_write(bit [11:0] a, bit [31:0] d);
        reg_transaction req, rsp;
        req = reg_transaction::type_id::create("wr");
        start_item(req);
        req.dir=reg_transaction::REG_WRITE; req.addr=a; req.wdata=d;
        finish_item(req); get_response(rsp);
    endtask

    task do_read(bit [11:0] a, output bit [31:0] v);
        reg_transaction req, rsp;
        req = reg_transaction::type_id::create("rd");
        start_item(req);
        req.dir=reg_transaction::REG_READ; req.addr=a;
        finish_item(req); get_response(rsp);
        v = rsp.rdata;
    endtask

    task body();
        bit [31:0] v;

        do_write(LOCK_ADDR, 32'h0);
        do_write(DATA_ADDR, 32'hDEAD_BEEF);
        do_read(DATA_ADDR, v);
        if (v !== 32'hDEAD_BEEF) `uvm_error("LOCK","Pre-lock write failed")
        else `uvm_info("LOCK","Pre-lock OK",UVM_MEDIUM)

        // 考点: LOCK=1，写入被忽略
        do_write(LOCK_ADDR, 32'h1);
        do_write(DATA_ADDR, 32'hCAFE_BABE);
        do_read(DATA_ADDR, v);
        if (v !== 32'hDEAD_BEEF)
            `uvm_error("LOCK",
                $sformatf("LOCK failed: exp=0xDEAD_BEEF got=0x%08h",v))
        else `uvm_info("LOCK","LOCK protection OK",UVM_MEDIUM)

        // 考点: LOCK=0，可再次写入
        do_write(LOCK_ADDR, 32'h0);
        do_write(DATA_ADDR, 32'h1234_5678);
        do_read(DATA_ADDR, v);
        if (v !== 32'h1234_5678) `uvm_error("LOCK","Post-unlock write failed")
        else `uvm_info("LOCK","Post-unlock OK",UVM_LOW)
    endtask
endclass
```